수능형 문제 풀이 모델 생성 WRAP-UP REPORT

1. 개요

1.1. 프로젝트 요약

- **프로젝트 주제** : '한국사'와 '수능 시험'에 특화된 모델 개발
- **프로젝트 목표** : 수능 국어 및 사회 과목의 복합적인 지문을 이해하고 추론하여, GPT-4, Claude, Gemini와 같은 초거대 모델을 능가하는 성능을 가진 경량화 모델을 구축하는 것을 목표로 함
- **주요 특징**
  - 수능 특유의 문제 구조(paragraph, problems, question_plus)에 최적화된 모델링.
  - 저작권 및 대회 규정을 준수하며, 수능 기출 데이터가 배제된 환경에서 효과적인 데이터 증강 기술(AI API 활용 등) 적용

1.2. 활용 장비 및 협업 환경

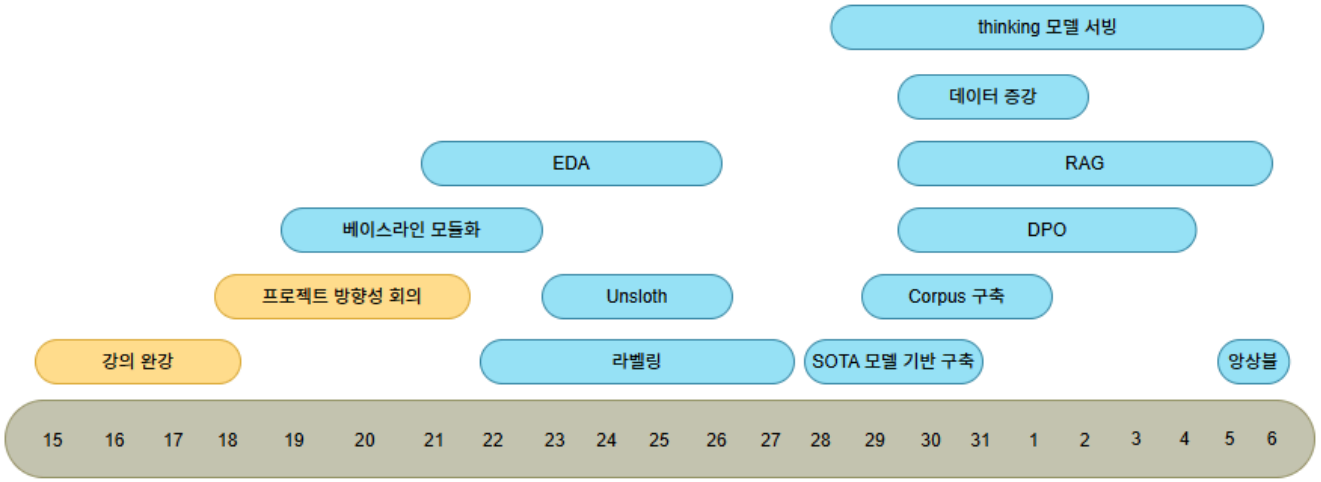
- **컴퓨팅 환경** : Linux 기반 원격 GPU(V100 32GB) 서버 3개를 SSH 접속을 통해 활용
- **협업** : GitHub, Slack, 카카오톡, Notion(문서화 및 일정 관리)
- **개발 프레임 워크** : Unsloth, HuggingFace Transformers, TRL, PEFT, Hydra

2. 프로젝트 팀 구성 및 역할

- 팀 NLP-09 (파란 부스터팀)

이름	역할
강다형	데이터 라벨링, 가상환경/패키지 설정, qwen3-thinking 모델 서빙, 스크럼 모더레이터
김세훈	데이터 라벨링, Rag corpus구축, 한국사 Rag 파이프라인 설계
박준범	데이터 라벨링, 베이스라인 모듈화, github 초기 셋팅, SFT실험
허승환	팀 notion 설계, EDA, 데이터 라벨링, DPO, CoT, 데이터 생성
김민석	모델 탐색 및 파이프라인, 데이터 증강 및 전처리, 데이터 분류 파이프라인, 프롬프트 모듈화
이태원	데이터 라벨링, 베이스라인 모듈화, 데이터 증강, 앙상블

3. 프로젝트 수행 절차 및 타임라인



3.1. 프로젝트 착수 및 기반 학습 (12/15 – 12/23)

- **강의 수강 및 프로젝트 마감 설정** : 12월 15일부터 12월 18일까지 강의를 완강하며 기초 역량을 확보하고, 전체 프로젝트 마감 일정 수립
- **전략 수립 및 베이스라인 구축** : 12월 19일 방향성 회의를 통해 수능형 모델 개발 전략을 확립했습니다. 코드의 재사용성과 협업 효율을 위해 베이스라인 모듈화를 완료하여 개발 기틀을 마련

3.2. 데이터 분석 및 초기 모델 실험 (12/24 – 12/31)

- **데이터 심층 분석** : 지문 구조 파악을 위한 EDA를 진행하고, 고품질 학습 데이터 확보를 위해 팀원 모두가 참여하여 데이터 라벨링을 완료
- **초기 성능 검증 및 최적화 학습** : 첫 리더보드 제출로 베이스라인 성능을 확인. 또한, 학습 효율을 극대화하기 위해 unsloth 패키지 활용하는 실험을 계획

3.3. 고도화 모델 구축 및 추론 검증 (01/01 – 01/04)

- **SOTA 모델 구현** : 최신 아키텍처를 리서치하여 Qwen2.5-32B-instruct-bnb-4bit 모델 기반 베이스라인을 구축하고, 본격적인 학습 및 추론 실험을 수행
- **RAG 시스템 및 서빙 테스트** : RAG Corpus 및 문제 분류 파이프라인을 구축하여 지식 기반 추론 성능을 보완. 특히 Qwen3-30B-Thinking 서빙을 통해 모델의 한국어 지식과 논리 구조를 직접 검증하며 서빙 가능성을 타진

3.4. 최종 최적화 및 앙상블 (01/05 – 01/06 19:00)

- **추론 성능 극대화** : 대회 막바지, Qwen3 계열의 CoT(Chain of Thought) 및 실험했던 모델과의 앙상블(Ensemble) 전략을 집중적으로 적용하여 모델의 문제 풀이 논리를 고도화
- **실험 관리 및 세부 조정** : 심볼 변경 등의 프롬프트를 변경하는 세부 실험을 진행하였으며, 데이터 전처리와 RAG 최적화를 지속 (※ DPO, LoRA 튜닝, 데이터 증강 등은 우선순위에 따라 보류 및 검토를 거침)
- **최종 제출 및 마무리** : LoRA 파라미터 튜닝과 심볼 변경 등의 세부 작업을 거쳐 1월 6일 19시 마감에 맞춰 앙상블 결과물을 최종 제출

4. 프로젝트 수행 과정 및 결과

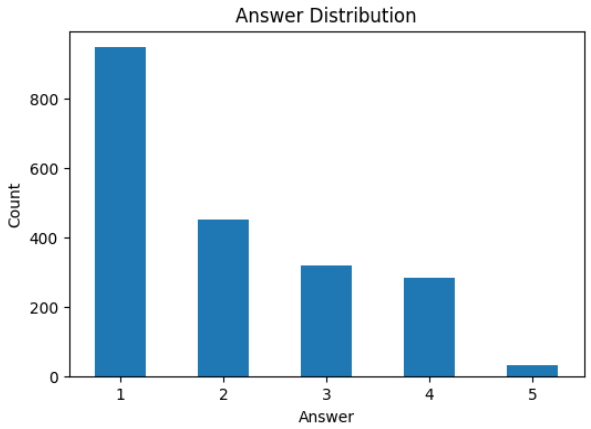
4.1. 베이스라인 모듈화

기존 단일 파일 기반 베이스라인은 빠른 실험에는 유리했으나 협업 및 반복 실험 환경에서는 구조적 한계가 있었기 때문에, 이를 개선하기 위해 설정과 로직을 분리하고 실험 관리 중심의 모듈화를 진행

- Registry 패턴과 Hydra 설정을 도입하여 모델·데이터셋 교체 및 실험 재현성을 확보
- 추상 클래스 기반 인터페이스 표준화를 통해 확장성과 협업 효율성 향상
- 베이스라인 성능을 유지하면서 모듈형 실험 환경 구축

4.2. EDA 및 데이터 라벨링

4.2.1. EDA 및 인사이트

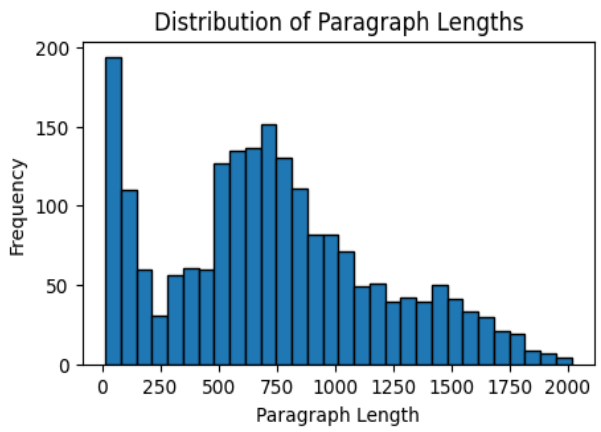


[그래프] dataset 전체 answer 분포

선택지	4지선다	5지선다
1	183 (23.11%)	765 (61.74%)
2	185 (23.36%)	265 (21.39%)
3	203 (25.63%)	116 (9.36%)
4	221 (27.9%)	62 (5%)
5	0	31 (2.5%)

[표] n지선다 answer 빈도 및 비율

- choices의 갯수는 4개 혹은 5개로 이루어져있으며, 4지 선다 문제의 경우에는 answer분포가 일정하나, 5지 선다의 분포의 경우는 answer의 불균형이 심함
  → 불균형을 해소하여 성능 비교 실험



[그래프] dataset 전체 paragraph 길이

컬럼	결측치 갯수
id	0
paragraph	0
choices	0
answer	0
question_plus	2031

[표] dataset 결측치 갯수

- paragraph 길이는 대부분 2000자 이내
  → 프롬프트 내용 및 RAG 고려하여 max_seq_length는 4096으로 제한하여 실험 진행
- paragraph가 긴 문제에서 thinking 모델을 사용하여 reasoning 실험 진행
- train set에서 question_plus는 컬럼만 존재하고 내용이 있는 샘플 없음
  → 만약 존재한다해도 프롬프트상에서 paragraph 아래에 붙여 진행

4.2.2. 데이터 라벨링

EDA를 수행하는 과정에서, 한국사 과목과 같이 문제 해결 시 외부 지식이 필수적으로 요구되는 문항을 별도로 식별하고 분류할 필요성이 대두되었기 때문에, 팀 내부 합의를 바탕으로 총 2,031개의 학습 데이터를 팀원 간 분배하여 라벨링 작업을 수행

index	유형	특징
0 – 72	한국사 문제	지문기반 추론 불가능
73 – 186	지문기반 추론 문제	지문기반 추론 가능
187 – 206	역사적 맥락 매칭	paragraph 잘린 경우가 많음
207 – 320	경제 기반 추론 문제	문제 풀이를 위한 정보가 있는 표, 그래프가 짤린 경우가 다소 많음
321 – 460	본문 기반 추론 문제	지문기반 추론 가능
461 – 516	세계사 배경 지식	지문기반 추론 불가능
519 – 605	년도 기반 비슷한 사건	선택지에서 비슷한 역사적 사건 찾기 문제
606 – 791	세계사 및 고대 역사	시대적 정보가 필요한 문제
792 – 2031	지문기반 사실 추출	지문기반 추론 가능, 대부분 parahraph가 긴 형태

- 라벨링 기준 및 결과

라벨	기준	갯수
-1	오류가 있는 문제	69
0	지문 기반 추론 가능 문제	1747
1	외부 지식이 필요한 문제	210
2	paragrpah나 question이 중간에 짤린 문제	5

  - 품질이 매우 떨어지는 샘플(표, 그래프 없음 및 문장 잘림, 답이 다르거나 문제 오류 등) 존재
    → 제외 후 성능 비교 실험 진행
  - 크게 한국사, 비한국사로 나뉘며 한국사 문제의 경우 외부 지식없이는 풀이가 불가능
    → 한국사 문제의 경우에만 외부 문서 삽입(RAG)하여 실험 진행
  - 문제 오류 유형
    - 지문과 선지가 매칭이 안되는 경우 : 4문제
    - 질문과 선지 둘다 문제가 있는 경우 : 2문제
    - 선지가 잘못된 경우 : 23문제
    - 질문이 잘못된 경우 : 3문제
    - 지문이 잘못된 경우 : 7문제
    - 답이 이상한 경우 : 오답 (9문제), 복수정답 (15문제), 애매 (3문제), 정답이 없음 (3문제)

4.2.3. 에러 케이스

- paragraph가 긴 문제에서 자주 틀림
- 한국사 문제에서 자주 틀림

4.2.4. 데이터 전처리

- 오류 있는 문제 제외 (데이터 라벨링 -1)
- answer 분포 리밸런스

  → 4지 선다 문제에 대해 '해당 없음' 이라는 선택지를 추가하여 5지선다 문제로 만든다음, 전체 데이터셋에 대하여 answer의 분포를 약 20%씩 고르게 분포 시킴

실험	Macro-F1
base	0.76
오류있는 문제(-1) 제거	0.7711
answer 분포 리밸런스	0.7734

[표] 데이터 전처리에 따른 F1 지표 변화

4.3. SFT

4.3.1 모델 선정

Scaling Law 이론에 따르면 동일한 데이터 분포와 학습 조건 하에서 모델의 파라미터 수가 증가할수록 성능이 향상되는 경향을 보이기 때문에, 본 프로젝트에서도 이러한 가설을 활용하여 모델 규모를 점진적으로 확장하는 방향으로 실험을 진행하였으며, 그 결과 대형 모델 기반 접근이 효과적임을 확인

- 사전 모델 실험
  - gemma-2b-ko 모델을 베이스라인으로 설정하여 inference 기반 실험 수행
  - 중형 모델인 Qwen2.5-7B 및 14B 적용 시 성능이 유의미하게 개선됨을 확인
  - 대형 모델을 대상으로 한 Fine-Tuning이 효과적일 것이라는 가설 수립
- 환경 제약 및 대응
  - Tesla V100(32GB) 환경에서 30B급 모델 활용에 메모리, disk 제약 존재
  - 대형 모델을 효율적으로 활용하기 위해 Unsloth 라이브러리 도입
- SFT 대상 모델 선정
  - **모델** : unsloth/Qwen2.5-32B-instruct-bnb-4bit
  - **선정 이유**
    - 4bit 양자화된 모델 사용을 통한 disk 사용량 절감
    - 제한된 연산 자원 환경에서도 대규모 파라미터 유지 가능
    - SFT 실험을 위한 SOTA 기준 모델로 적합하다고 판단

4.3.2 SFT 수행

선정된 대형 모델을 대상으로 태스크 특화 성능 향상을 목표로 Supervised Fine-Tuning을 수행.
학습 효율성과 메모리 사용량을 고려하여 파라미터 효율적 미세조정 기법을 적용하였으며, 학습 안정성을 높이기 위한 설정 개선을 병행

- 학습 방식
  - 파라미터 효율적 학습을 위해 LoRA(Low-Rank Adaptation) 사용
- 학습 설정
  - Unsloth 기반 메모리 최적화 활용으로 기존 대비 더 큰 batch size 확보
  - max sequence length 증가를 통한 장문 문맥 반영
- 학습 결과
  - 태스크에 특화된 표현 학습 가능
  - 단순 모델 크기 증가를 넘어 실질적인 성능 개선 확인 (0.7171 → 0.7600)
- 종합 평가
  - Scaling Law 기반 대형 모델 선정과 SFT 전략의 유효성 검증
  - 제한된 연산 자원 환경에서도 Unsloth + LoRA 기반 SFT가 효과적인 접근법임을 확인

4.4. 데이터 증강

데이터 라벨링 완료 후 학습 데이터셋(Train Dataset)의 품질 확보에 대한 필요성이 제기되었으며, 전처리 과정에서 발생할 수 있는 데이터 수량 감소 문제에 대비하고자 하였다. 이를 해결하기 위해 외부 데이터셋 활용과 Self-instruct을 병행하여 데이터 증강을 수행

4.4.1. 외부 데이터셋 활용

- HAERAE-HUB/HAE_RAE_BENCHMARK_1.0
  - 본 대회의 KMMLU 데이터셋을 공개한 HAERAE-HUB의 다른 데이터셋인 HAE_RAE_BENCHMARK_1.0(허깅페이스)을 활용
  - 전체 데이터 중 수능 태스크와 문제 유형이 유사한 '독해' 및 '한국사' 영역만을 선별하여 학습에 포함
- 국가직 공무원 9급 한국사 5개년(2021-2025) 기출 데이터
  - 수능 한국사 과목과 가장 유사한 문제 형식인 국가직 공무원 9급 한국사 데이터셋을 추가로 활용하여 실험을 진행
  - 최근 5개년 치의 기출문제를 수집하였으며, 연간 20문항씩 총 100건의 데이터를 구축

4.4.2. Self-instruct 기반 데이터 증강

- LLM API를 이용해 데이터를 생성하는 self-instruct 방법을 이용
- 데이터 라벨링 단계에서 분류된 문제 유형의 비율을 반영하여, 균형 잡힌 50개의 문항으로 Seed Data를 구성
- OpenAI의 GPT-4o 모델을 활용하여, Seed Data를 바탕으로 총 500건의 데이터를 증강

데이터셋	Macro-F1
대회 train set	0.7600
+HAE_RAE(독해)	0.7620
+HAE_RAE(독해, 한국사)	0.7644
+공무원 한국사	0.7624
+self_instruct	0.7719
+HAE_RAE(독해, 한국사), 공무원 한국사,self_instruct	0.7724

[표] 데이터셋 구성에 따른 F1 지표 변화

4.5. Thinking Model

기본적인 파이프라인을 완성한 이후 팀 내 모델 리서치를 통해 qwen 계열 모델을 사용하기로 결정했고 파라미터 수가 30B 가량인 모델 중, MMLU-Pro 벤치마크 점수가 가장 높은 Qwen3-30B-A3B-Thinking-2507 모델을 선택.
서버 환경에서 모델을 띄우기 위해 Unsloth Dynamic GGUF 양자화 기법이 적용된 모델을 사용했으며 ollama 패키지를 통해 서빙

데이터셋 출처

- HAE_RAE_BENCHMARK_1.0
  huggingface, HAERAE-HUB, https://huggingface.co/datasets/HAERAE-HUB/HAE_RAE_BENCH_1.0
- 국가직 공무원 9급 한국사 5개년(2021-2025) 기출 데이터
  사이버국가고시센터, 9급 공채, https://www.gosi.kr/

4.5.1. think 옵션 미사용

ollama api에서 structured output format 옵션을 사용할 경우 think가 작동하지 않는 이슈 발생으로 4.5.1은 이슈를 해결하기 전에 진행한 실험에 대해 서술

- **실험1 – thinking 모델 기준**
  - best practice 파라미터 값을 설정했을 때 리더보드상 Macro F1 score는 0.7598을 기록 (step7)
  - 이 값을 기준으로 think 옵션이 활성화 된 경우의 모델 성능 변화를 측정

- **실험2 – 전처리한 train 데이터를 eval 데이터로서 활용**
  - MoE 모델에서 SFT를 진행하는 것은 기술적으로 까다롭기 때문에 시도하지 않음
  - 따라서 train 데이터를 eval 데이터로서 활용하여 분석을 수행

지표	값
Accuracy	86.98% (1703/1958)
Macro F1-score	0.8317

[표] think 옵션 미사용 평가 지표

클래스	Macro-F1	Support
1	0.9300	928
2	0.8510	433
3	0.8136	309
4	0.7780 (최저)	264
5	0.7857	24

[표] 추론 시 train set을 eval set으로 활용한 F1 평가 지표

- train과 test 데이터의 0.0719의 Macro-F1 점수차
  → 이를 근거로 test 데이터의 문제가 모델이 풀기 더 어려운 문제일 것이라고 판단
- 클래스별 F1-score가 상이하며 1번 선지(최다 클래스)와 4번 선지의 차이가 가장 큼
  → EDA 결과 에러 케이스 분석을 참고하여 SFT에서는 객관식 선지 번호 셔플을 적용하여 특정 번호에 대한 bias를 제거하고자 했다.

4.5.2. think 옵션 사용

4.5.1의 이슈를 해결한 후 진행한 실험에 대해 서술. best practice에 따라 파라미터를 설정했고, context_length 파라미터는 프롬프트 최대 토큰 수와 num_predict 합을 넘지 않도록 설정하여. 하단의 총 3가지 실험을 진행

- **실험 1 – 간단한 프롬프트**
  단순히 지문과 문제를 제시한 후 '선택지 가짓수' 이하의 자연수를 출력하도록 지시
  test 데이터에 대한 Macro-F1 점수는 0.8076을 기록 (step7)

- **실험 2 – 중간 복잡도 프롬프트**
  - CoT가 정답을 잘 도출할 수 있도록 유도하고자 시스템 프롬프트에 다음과 같은 문제 풀이 규칙을 제시
  - EDA를 통해 분석한 문제 유형 키워드를 시스템 프롬프트에 추가
  - 지문, 질문, 선택지를 주요 근거로 활용할 것을 요구
  - 외부 지식이 필요하다고 판단되면 이를 활용해서 정답을 고르는 근거로 활용할 것을 요구
  - 까다로운 추론이 필요한 경우 깊게 생각할 것을 요구하고 지문, 질문, 선택지를 재확인할 것을 요구
  - 최종 출력은 선택지와 대응되는 하나의 숫자만을 출력하도록 요구

- **실험 3 – 높은 복잡도 프롬프트**
  팀에서 모든 train 데이터에 대한 라벨링을 진행하며 파악한 데이터 특성을 프롬프트 규칙에 반영하고자 실험 2의 프롬프트에서 추가적으로 문제 유형 가이드를 제시한 후, 문제를 풀도록 유도
  제시했던 문제 유형은 다음과 같다.
  - 지문을 근거로 풀 수 있는 문제
    - 쉽게 풀 수 있는 문제
    - 까다로운 추론이 요구되는 문제
  - 지문 외 외부 지식이 필요한 문제
    - 외부지식에 해당하는 키워드를 함께 제시

4.6. RAG

4.6.1 문제 정의 및 접근 전략

- 세계사 영역의 정답률은 우수
- 한국사 영역(약 70건)에서는 대부분 모델이 예측을 틀린 경향을 확인

오답 분석 결과, 모델이 한국사 도메인에 대한 세부정보가 부족하다고 판단되어 이를 RAG 파이프라인을 통해 해결 시도

4.6.2 corpus 구성

데이터의 신뢰성을 확보하고 외부 웹 크롤링 시 발생할 수 있는 노이즈를 배제하기 위해 자체 구축방식을 채택

- **Corpus의 분류 체계를 7대 카테고리로 분류**
  train 데이터셋(한국사 70건)을 분석한 결과 밑줄 친 '왕', '인물', (가) 인물 ,다음 자료, 다음 선언문 등의 유형이 다수임을 파악하여, Corpus의 분류 체계를 7대 카테고리(통치자, 주요인물, 사건/전쟁, 단체/기구, 서적/문화유산, 제도/법령, 생활상)로 구조화
- **Title(주제) 선정 및 분류는 사람이 직접 수행하여 정확도를 높임**
- **Content는 대형 LLM을 활용하여 생성**
- **최종적으로 280건의 고품질 문서를 확보**
- **검색 정확도를 극대화 하기 위해서 스키마를 정교하게 설계**
  - doc_id & category : 데이터 관리 및 필터링 용도
  - title & aliases : 동의어(예: 세종대왕, 이도) 처리를 통한 Keyword 검색
  - rag_matching_keywords & search_text : 검색엔진이 인덱싱할 전용 필드
  - context : LLM에게 제공될 실제 배경지식

4.6.3 Retriever 실험

Train 데이터 분석 결과 문맥적 추론과 정확한 키워드 매칭이 동시에 요구됨을 확인하여, FAISS(Vector)와 BM25(Keyword)를 결합한 Hybrid Retrieval 파이프라인을 구축

1. 임베딩 모델로 text-embedding-3-small을 채택하여 문맥 유사도를 확보
2. BM25는 속도 최적화를 위해 별도의 형태소 분석 없이 단어 단위 토크나이징을 적용

- 구조
  1. 각 검색기에서 상위 10개 문서를 추출
  2. 가중합 방식을 통해 재정렬하여 최종 Top-5 문서를 LLM에 전달
- 성능 평가 결과
  - HAERAE 및 공무원 기출 데이터셋 총 289문항 평가
  - 222문항 정답으로 RAG 미적용한 Base 모델 대비 유의미한 성능 향상 확인
- 오답 원인 분석
  - **Corpus 누락** : 약 70%
  - **검색 실패** : 키워드 불일치로 인한 BM25 매칭 실패
  - **프롬프트 추론 오류** : 4건
- 고도화 수행 (개선 사항)
  - 지엽적 개념 및 사료 데이터 추가로 Corpus 확장
  - Alias(별칭) 및 관련 용어 보강으로 BM25 검색 정확도 개선
  - 검색 문서를 우선 활용하도록 프롬프트 구체화

4.6.4 한국사 문제 분류

Test 데이터셋의 문제 유형 혼재로 외부 LLM 기반 분류에는 한계가 있었으며, 정확도·비용 효율 측면에서 자체 학습 모델을 활용한 2차 분류 방식을 적용

- **1차 분류** – 한국사 내용 포함 여부 분류 (한국사/비한국사)
  학습에 사용한 동일 모델을 사용하여 프롬프트로 분류
- **2차 분류** – 외부 지식(문서) 필요 여부 분류
  2차 분류의 목적은 지문 정보만으로 해결 가능한 문제를 선별하는 데 있고, 이를 위해 동일 모델을 사용해 선택지 판단 시 지문 기반 추론 가능 여부를 기준으로 추가 분리를 수행
- **결과 및 한계** – 한국사 문제 중 약 85%의 정확도로 외부 문서 삽입. 하지만 여전히 소수의 비한국사 샘플에도 외부문서가 삽입되는 노이즈가 존재

4.7. Ensemble

- 최종 결과 도출을 위해 test set에 대한 각 모델의 추론 결과를 다수결로 결합하는 Hard Voting 방식을 사용하였다.
- 투표 결과가 동점인 경우를 대비하여, 단일 모델 성능 평가에서 가장 우수한 성과를 보였던 qwen3_thinking 모델의 예측값에 우선순위(Casting Vote)를 부여하여 최종 정답을 결정
- RAG 적용시에 한국사 성능은 향상되었으나 비한국사 문제에 노이즈가 섞여서, 이를 해결하기 위해 RAG 파라미터 튜닝에서 성능이 우수했던 RAG 모델 2개와 기존 SOTA 모델을 결합한 앙상블을 진행. 한국사로 분류된 문제에 한해서만 RAG모델 두 개가 동일한 답을 가리킬때만 RAG 답안으로 수정한 결과, private 성능이 기존 SOTA보다 미세하게 올라 이 것을 최종 제출로 진행

4.8. 결과

단계	적용 기법	리더보드 Macro-F1	비고
Step 1	Baseline (Simple SFT)	0.2724	초기 기준점 확보
Step 2	qwen2.5실험 -7B-inference-only	0.5877 (+0.3153)	2B에서 7B모델 이용
Step 3	unsloth/Qwen2.5-32B-Instruct-bnb-4bit	0.7171 (+0.1394)	inference-only
Step 4	step3에서 max_length 증가 (2048 → 4096)	0.7477 (+0.0306)	
Step 5	unsloth/Qwen2.5-32B-Instruct-bnb-4bit	0.7600 (+0.0123)	SFT 적용
Step 6	step5에서 haerae dataset 추가	0.7744 (+0.0144)	독해+한국사
Step 7	unsloth/Qwen3-30B-A3B-Thinking-2507-GGUF	0.7598 (-)	Qwen3 적용
Step 8	step 7에서 think 모드 적용	0.8076 (+0.0179)	
Step 9	Qwen3 2개, Qwen2.5 2개, EXAONE으로 hard voting	0.8122 (+0.0046)	앙상블
Step 10	Sota + rag ensemble	0.8129 (+0.0007)	rag부분 추가 앙상블

5. 프로젝트 회고

5.1. 잘했던 점

- 체계적인 협업 인프라
  - 노션(Notion)의 보드와 타임라인 뷰를 활용하여, 3주간의 짧은 기간 동안 팀원 간의 태스크 중복 없이 효율적으로 진행 상황을 공유했습니다.
  - GitHub flow를 활용하여 git 버전관리, 팀원 간의 이슈 공유가 활발하게 이루어졌습니다.
  - 모듈화된 베이스라인 구축을 통한 실험 재현성 및 협업 효율성을 확보했습니다.
- EDA : EDA의 중요성을 인지하고 시간을 많이 들여 데이터 라벨링을 진행해 데이터에 더 친숙해질 수 있었습니다.
- 돌발 상황 대처: 초기 계획이 약간 틀어졌으나 빠른 역할 재분배를 통해 프로젝트를 안정화했습니다.

5.2. 시도했으나 잘 되지 않았던 것들

- **하이퍼파라미터 최적화** : LoRA 파라미터 튜닝 등의 세부 최적화 작업이 마감 직전까지 '진행 중' 혹은 '보류' 상태로 남아 있어, 모든 실험 조합을 완벽히 테스트하지 못한 아쉬움이 있습니다.
- **데이터 라벨링** : 시간을 들여서 모든 dataset에 대해 라벨링을 진행하였으나, 거기에서 많은 인사이트는 얻지는 못했고 실험으로 이어지지 않았습니다.
- **한국사 문제 분류** : test 데이터셋에서 실제 한국사만 분류하는데에 여러 시도를 했지만 다른 국어문제가 껴있는듯 정확한 분류를 진행하는데에 한계가 있었습니다.
- DPO : reasoning 즉 생성 기반으로 진행해야되지만 logit기반 학습되었단 걸 늦게 파악해서 제대로 적용을 못했습니다.

5.3. 아쉬웠던 점들

- **시간 배분의 제약** : 프로젝트 후반부에 DPO와 RAG 실험이 집중되면서, 각 기법의 독립적인 기여도를 정밀하게 분석하기 위한 ablation study 시간이 부족했습니다.
- **앙상블 전략의 다변화** : 마감 기한인 1월 6일 19시에 맞추어 최종 제출을 진행하느라, 더 다양한 가중치 조합의 앙상블 실험을 시도해보지 못한 점이 아쉽습니다.

5.4. 프로젝트를 통해 배운 점

- **데이터 중심 AI(Data-centric AI)의 중요성** : 모델의 파라미터 수도 중요하지만 학습 데이터의 품질과 수능 지문 구조에 최적화된 데이터 포맷팅이 성능도 큰 영향을 미친다는 점을 확인했습니다.
- **제한된 환경 활용** : unsloth를 이용해서 제한된 환경에서 큰 모델을 효율적으로 이용할 수 있었습니다.

6. 개인 회고

강다형

이전 프로젝트의 진행 흐름이 '각자 해보고 싶은 것을 자유롭게 하며 공부해보자.'였다면, 이번 프로젝트의 팀 목표는 '팀끼리 좋은 협업을 하자.'였다고 생각합니다. 개인적인 목표는 크게 두 가지였습니다. 첫째, 팀원으로서 좋은 협업을 만들어 가는 과정에 최대한 기여하는 것입니다. 둘째, 제가 팀 내에서 담당한 파트를 명확히 이해하고 팀과 공유하는 것입니다. 특히 '좋은 협업'을 어떻게 해야 할지에 대해 많이 생각해봤습니다. 직전 프로젝트와 달리 팀 차원에서 필요하다고 생각했던 일들(가상환경/패키지 설정, 스크럼 모더레이터 등)을 먼저 찾아서 수행하고자 노력했습니다. '좋은 협업'에 대해 많이 고민했음에도 불구하고, 정작 팀과 '진정으로 대화'하는 시간을 충분히 갖지 못한 것 같아서 아쉽습니다. 노션, git 버전 관리, 모듈화 등 협업을 위한 툴이나 기술은 팀에 안정적으로 도입됐다고 생각합니다. 그러나 그보다 더 중요했던 것은 6명이 함께 나아가고자 하는 방향을 지속적으로 점검하고 조정하는 과정이었던 것 같습니다. 다음 프로젝트에서는 서로가 어떤 생각을 하는지, 무엇을 원하는지 조금 더 많이 이야기해 보면 좋을 것 같습니다. 다른 사람들 눈에 화려해보이지 않을지라도 우리 팀만의 의도가 담긴 결과물을 만들고 싶습니다.
회고를 작성하며 새삼 제가 좋은 팀에 속해 있음을 다시금 깨닫습니다. 팀원들에게 고마움을 표하며 회고를 마칩니다.

김민석A

이번 프로젝트에서는 LLM을 사용하면서 서버에서 사용 가능한 최대한 큰 모델을 쓰는것이 우선순위라고 판단해서 어떤 모델이 서버내에서 돌아갈수있을지 disk 용량적인 면이나 vram 면에서 매우 많이 찾아보았고 원본 모델은 disk 용량이 너무 커서 사용할수 없음을 판단. 용량이 작은 양자화된 모델 위주로 찾아보았으며 그 중에서 버그가 없고 제일 안정된 qwen2.5계열의 instruct 튜닝된 모델을 선택하게 되었음.

추가적으로 이 프로젝트에서 학습적인면으로 추론만 가능한 모델(A3B, MoE 등)보다는 학습까지 가능한 모델을 찾는것을 우선순위로 두었습니다.

다만, 후보 모델을 더 효율적으로 서버에서 사용하기위해 무작정 unsloth를 사용하였는데 패키지 버전이 충돌한다던가, 의존성 문제가 많이 발생해서 시간을 많이 빼앗겼기 때문에 앞으로는 새로운 프레임워크 사용전에 패키지의 의존성부터 확인하는 습관을 가져야 할 교훈을 얻었습니다.

또한 EDA를 진행하면서 answer의 정답 분포가 균등하지 않다는점, 프롬프트내에서 선택지의 심볼을 숫자(1-5)로 주는것에 편향이 있을것이라는 가설을 세우고 실험을 진행했는데, answer의 정답분포를 균등화 시키면서 성능이 향상됐으나, 심볼을 변경하거나 choices의 순서를 섞는등의 실험에서는 유의미한 성능 향상을 이루어내지 못했습니다. 해당 가설을 세우고 논문도 참고하였는데 유의미한 결과를 거두지 못했기때문에 마지막 최종 프로젝트에서 이러한 편향같은것들을 좀 더 알아보고 실험해볼것같습니다.

두번째로 LLM으로 한국사 문제 및 외부 문서 판단 여부를 분류하는 작업을 진행하였는데, 처음에는 휴리스틱한 방법으로 분류를 하고 LLM으로 진행해보려고 하였으나 데이터를 증강시키며 paragraph,와 question의 다양성이 증가해서 휴리스틱한 방법으로는 어려울것이라고 판단해서 LLM만 사용. 초기에는 한국사여부 + 외부문서 필요를 한번의 프롬프트로 분류시키게 하였으나 기준이 2개라 모델이 경계선을 정하는데 헷갈려 해서 한국사 문제 여부를 분류하는 1차 프롬프트와 외부 문서를 필요로하는 2차 프롬프트로 나누어 진행했습니다. 프롬프트 개선으로는 한국사보다는 비한국사문제에 외부문서가 삽입되는것이 노이즈가 더 클것이라고 판단하여 그 부분을 중점으로 정확도가 높아지게 프롬프트를 개선시켜 나가 dataset에서 85~90%의 정확도로 한국사문제를 분류해내는 결과를 냈습니다.

저번 프로젝트에서는 깃허브를 제대로 활용하지 못했기때문에 코드 정리나 실험을 진행하면서 진행상황 공유나 나의 브랜치에 제때제때 merge하지 않아서 곤란을 많이 겪어서 이번 프로젝트에서는 최대한 깃허브의 issue와 pr 활용 및 merge를 받음으로써 버전관리가 이전 프로젝트에 비해서 많이 개선됐습니다.

김세훈

이번 프로젝트는 지난 협업의 아쉬움을 보완하고자 코드 모듈화와 GitHub 워크플로우 정착에 주력했습니다. 팀원들과 오프라인 회고를 병행하며 협업 효율을 높였으나, 후반부 병렬 실험 과정에서 파이프라인 수정과 패키지 버전 관리가 다소 느슨해진 점은 아쉬움으로 남습니다.

데이터 측면에서는 1주일간 1,000건의 라벨링을 진행하며 데이터의 이해도를 높였으나, 모델의 내재 지식과 인간의 판단 기준 차이로 인해 이를 실험에 온전히 활용하지 못한 점이 아쉬웠습니다. 또한 RAG 설계를 주도하며 평가 지표 수립의 어려움과 도메인 분류의 모호함, max_token 설정 등 기술적 난관을 겪었고, 타 팀의 LangFuse 활용을 보며 체계적인 실험 추적 관리의 중요성도 절감했습니다.

하지만 이 모든 과정은 성장의 밑거름이라 생각합니다. Raw 데이터를 집요하게 분석하는데에 성능 병목을 찾는 시야를 길렀습니다. 또 Rag를 밑바닥부터 error_case및 데이터 분석을 통해서 설계했고, 또 자체적인 데이터 분석을 통한 자체적인 Corpus 구축과 Hybrid Retrieval 파이프라인 설계를 통해 베이스라인 대비 유의미한 성능향상을 이끌어 낸 점은 가장 큰 성과였습니다. 이러한 시행착오를 통해 다음은 더 발전해서서 유지보수 가능한 엔지니어로 나가자고 합니다. 이를 위해 Python의 심화 문법과 디자인 패턴을 체화하여 코드의 구조적 밀도를 높이고, 강의에서 다룰 Airflow와 Docker를 도입해 패키지 관리부터 Serving 단계까지 고려한 확장성 있는 파이프라인을 구축하고자 합니다. 또한 이번 프로젝트에서 아쉬웠던 실험 관리와 로깅 체계를 보완하기 위해 LangFuse와 같은 MLOps 도구를 학습하여 실험관리나 효율을 높이고자 합니다. 다가올 최종 프로젝트에서는 단순히 기능 구현에 급급하기보다, 문제 정의부터 서비스 배포까지 논리적 완결성을 갖춘 파이프라인을 완성하는 자세로 목표하고자 합니다.

박준범

이번 프로젝트의 핵심 목표는 모델 성능의 불확실성을 제거하고, 팀원 누구나 일관되게 실험을 재현할 수 있는 기반을 구축하는 것이었다. 또한 모델의 추론 능력을 올리기 위해 데이터 기반의 동적 추론 방식을 도입하는 데 집중했다.

먼저 모듈화 시 성능 하락이 발생해 원인을 찾기 위해 Baseline과 모듈화 코드를 라인 단위로 대조 분석했다. 이 과정에서 프롬프트 문구 차이, 토큰 유실, 필수 태그 누락 등의 결함을 수정하여 Baseline의 동작을 재현했다. 학습 효율을 높이기 위해 Unsloth 라이브러리를 통합하고 안정적인 학습 환경을 마련했다.

특히 모델의 지능을 보완하기 위해 Dynamic Few-shot 기능을 실험 과제로 삼고 구현했다. 모든 문제에 동일한 예시를 주는 방식은 지문 도메인에 따라 오히려 방해가 될 수 있다고 판단했다. 이를 해결하기 위해 OpenAI text-embedding-3-small 모델을 사용하여 학습 데이터 전체를 벡터화하고 FAISS 인덱스에 저장했다. 추론 시점에 테스트 문제와 가장 유사한 학습 데이터를 실시간으로 검색하여 프롬프트에 주입하는 기능을을 구축했다.

이전 프로젝트와 달리 이번에는 지속 가능한 실험 환경을 구축하는 방식을 변화시켰다. 코드는 고정하고 설정만 변경한다는 원칙 아래 시스템 구조를 설계했다.

구체적으로 모든 모델과 실험 설정을 하드코딩하지 않고, MODEL_REGISTRY와 Hydra 기반 설정 파일을 통해 관리하도록 객체 지향적 구조를 도입했다. 이로 인해 팀원들은 소스 코드를 수정하지 않고도 unsloth_config.yaml 등 설정 파일 변경만으로 동일한 실험을 재현할 수 있었다. 그 결과 협업 과정에서 발생하던 코드 충돌이 현저히 줄었고, 실험 결과에 대한 팀 내 신뢰도와 재현성이 크게 향상되는 효과를 얻었다.

기술적 고도화를 시도하며 명확한 한계도 확인했다. Dynamic Few-shot 방식이 실제 리더보드 점수 향상으로 직결되지 못한 점이 가장 큰 아쉬움으로 남는다. 검색된 예시가 모델의 정답 도출에 결정적인 힌트를 주지 못했거나, sLM의 컨텍스트 처리 능력 한계로 인해 추가된 정보가 오히려 혼란을 주지 않을까 생각했다다.

또한 학습된 모델을 GGUF 형식으로 변환하여 Ollama와 LangChain을 연동한 멀티 에이전트 시스템까지 나아가려 했으나, 실제 구현은 SFT 학습과 기본적인 추론 검증 단계에 머물렀다.

run.py라는 단일 파일에 학습, 추론, 평가 로직이 집중되어 프로젝트 후반부에 새로운 기능을 추가할 때마다 유지보수가 어려워졌던 설계 상의 미흡함도 인식하게 되었다.

프로젝트를 통해 성능 개선을 위한 기술 도입만큼이나, 해당 기술이 모델의 특성과 데이터에 적합한지 정밀하게 검증하는 과정이 중요하다는 교훈을 얻었다. 또한 확장 가능한 아키텍처 설계가 장기적인 생산성에 미치는 영향을 체감했다.

다음 프로젝트에서는 학습, 추론, 평가 파이프라인을 완전히 분리하고, 공통 인터페이스를 공유하는 플러그인 형태의 모듈 구조를 설계할 것이다. 이번에 시도했던 Dynamic Few-shot의 실패 원인을 분석하여, 단순히 유사한 문제를 찾는 것을 넘어 모델이 정말로 필요로 하는 정보를 선별하여 제공하는 더 정교한 RAG 시스템을 구축하고 싶다. 나아가 Ollama와 LangChain 에이전트를 도입해 복합적인 사고를 수행하는 추론 엔진 구축에도 다시 한번 도전하겠다.

이태원

지난 프로젝트 회고를 통해 협업 프로세스와 코드 통합의 어려움이 프로젝트의 가장 큰 병목임을 인식했습니다. 따라서 이번 프로젝트의 최우선 목표는 '지속 가능한 협업 환경 구축'으로 설정했습니다. 이를 위해 깃허브와 노션을 도입하여 이슈 트래킹을 활발히하고, 매일 데일리 스크럼과 피어 세션을 통해 진행 상황을 공유하며 소통의 부재를 해결하고자 노력했습니다.

기술적으로는 팀원 전체의 생산성을 높이기 위해 베이스라인 코드의 모듈화를 주도했습니다. 하이드라 프레임워크를 도입하여 복잡한 하이퍼파라미터를 Config 파일로 관리하게 함으로써, 팀원 누구나 타인의 실험을 손쉽게 재현하고 검증할 수 있는 환경을 만들었습니다. 이 과정을 통해 잘 짜인 모듈 구조가 팀 전체의 연구 효율성에 얼마나 큰 영향을 미치는지 체감할 수 있었습니다.

모델 성능 향상을 위해서는 다양한 데이터 증강 실험을 수행했습니다. 우선 HAE-RAE 데이터셋 활용 시, 6개 영역 전체를 사용하는 대신 수능 문제와 유사한 '독해'와 '한국사' 영역만을 선별하여 학습시켰을 때 더 좋은 성능을 보임을 확인했습니다. 이는 무조건적인 데이터 증량보다 도메인 적합성이 높은 고품질 데이터를 선별하는 것이 중요함을 보여주었습니다. 반면, 공무원 한국사 데이터를 활용한 실험에서는 LoRA 파인튜닝만으로는 모델에 특정 지식을 주입하는 데 한계가 있음을 확인했습니다. 또한 GPT-4o를 이용한 Self-instruct 방식에서는 Seed data를 유형별로 균형잡히게 구성했음에도 생성된 데이터가 독해 유형으로 편향되는 문제를 겪으며, 향후 유형별 균형을 맞추기 위한 정교한 프롬프트 엔지니어링의 필요성을 느꼈습니다.

마지막으로 앙상블 과정에서는 모델 간 출력 방식의 차이(Generate vs Logit)로 인해 Hard Voting 방식을 채택했습니다. 이 과정에서 각 모델의 강점을 확인하기 위해 검증 데이터셋에 대한 각 모델의 에러케이스 분석을 진행하였습니다. 처음에는 thinking 모델은 긴 텍스트에 대해 정답률이 높고 sft를 진행한 qwen2.5 모델은 한국사같은 문제에 정답률이 높을 것으로 예상을 하였지만 실질적으로는 그러한 패턴을 찾지 못하여 앙상블이 좋은 성능 개선 효과를 보지 못했습니다.

협업 도구의 도입과는 별개로, 프로젝트 운영 측면인 일정 관리에서 뚜렷한 한계를 마주했습니다. 구체적인 팀 마일스톤 설정이 미흡했던 탓에 예상치 못한 변수 발생 시 일정이 지연되거나, 분업된 역할에 대해 팀원 간 소통 오류가 발생하여 초기 계획대로 작업이 진행되지 못하는 혼선이 있었습니다. 이를 통해 도구의 도입만큼이나 이를 운용하는 명확한 계획과 중간 점검이 필수적임을 깨달았습니다. 따라서 다음 프로젝트에서는 초기 기획 단계에서 전체 로드맵을 더 정교하게 설계하고, 주기적인 마일스톤 달성 여부를 점검하여 일정 리스크를 최소화하는데 더 힘을 쓰고 싶습니다.

결과적으로 이번 프로젝트를 통해 얻은 가장 큰 수확은 '협업의 가치'에 대한 재발견입니다. 서로 다른 가설과 실험 방향을 하나로 조율하는 과정은 혼자 작업할 때보다 훨씬 많은 에너지를 필요로 했습니다. 하지만 팀원들과 끊임없이 소통하며 하나의 목표를 향해 나아가며 시너지를 발휘할 수 있었고 성공적으로 프로젝트를 마무리할 수 있었습니다. 앞으로도 기술적인 깊이뿐만 아니라, 팀의 성장을 견인할 수 있는 소통 능력과 협업 마인드를 갖춘 엔지니어로 성장하고 싶습니다.

허승환

이번 프로젝트에서는 이전 프로젝트에서의 아쉬움을 보완하기 위해, 다양한 방법을 빠르게 적용해보는 것을 개인 목표로 설정하였습니다. 이를 위해 CoT, DPO, ORPO, 다회차 추론, 대조적 프롬프팅, Test-time Scaling, PriDe 기반 편향 제거 등 여러 기법을 조사하였고, 그중 비교적 적용이 가능하다고 판단한 DPO를 우선적으로 시도했습니다.

실험 속도를 높이기 위해 생성형 AI를 활용하여 학습 코드를 작성하고, 기존 train.csv를 기반으로 데이터를 생성해 추가 학습을 진행했습니다. 그러나 학습 과정에서 loss가 0으로 수렴하는 비정상적인 현상이 발생하였고 파라미터 설정, 데이터 분포 중복으로 인한 과적합, loss 정의 오류 등 여러 가능성을 고려하였으나, 마감 일정이 촉박한 상황에서 충분한 원인 분석을 진행하지 못하고 방향을 수정하게 되었습니다.

멘토링과 팀원들과의 논의를 통해, 현재 모델이 logit 기반 구조라는 점을 명확히 인식하게 되었고, 이로 인해 DPO 적용이 추론 과정(reasoning)의 개선보다는 정답 선택 확률을 높이는 데에만 기여할 수 있다는 구조적 한계를 이해하게 되었습니다. 이에 따라 생성 단계를 추가하고 CoT 기반 접근으로 방향을 전환했습니다.

CoT 적용을 위해 간단한 프롬프트를 구성하고, reasoning 데이터는 기존에 생성한 데이터 일부를 활용하였으며 코드도 일부 수정했습니다. 다만 프로젝트 후반부에는 서버 자원이 제한적이었고, 마감 직전에 확보된 시간 또한 충분하지 않아 실험 결과를 명확히 검증하지는 못했습니다.

결과적으로 성능 향상에는 큰 기여를 하지 못했으며, 주요 기여는 간단한 EDA 수행, 데이터 라벨링을 통한 데이터 구조 이해, 정답 분포 편향 및 4·5지선다 혼재 문제를 확인한 뒤 선지 수 통일 및 shuffle을 적용한 전처리 작업이었습니다.

이번 프로젝트를 통해 기법을 빠르게 적용하는 것만으로는 의미 있는 결과를 얻기 어렵고, 모델 구조와 학습 원리에 대한 이해가 선행되어야 한다는 점을 명확히 인식하게 되었습니다. 또한 생성형 AI를 활용한 코드 작성은 효율적일 수 있으나, 코드의 동작 원리를 이해하지 않은 상태에서는 문제 발생 시 적절한 판단과 대응이 어렵다는 한계도 경험하였습니다.

협업 측면에서는 프로젝트 초반 오프라인 회의를 통해 전체 방향과 GitHub 구조를 논의하며 출발하였으나, 후반부에 PR과 issue 기반 협업 과정에서 타인의 코드를 충분히 이해하지 못해 적극적인 리뷰와 의견 제시에 한계가 있었습니다. 이를 통해 코드 독해 능력과 GitHub 협업 경험을 더욱 강화할 필요성을 느끼게 되었다.

종합적으로 이번 프로젝트는 성능 측면의 성과보다는, 나의 현재 역량과 부족한 부분을 명확히 인식하는 계기가 되었습니다. 이를 바탕으로 최종 프로젝트에서는 새로운 기법을 무리하게 시도하기보다, 모델 구조와 코드 흐름을 정확히 이해하고 작은 단위라도 확실한 기여를 하는 방향으로 프로젝트에 임하고자 합니다.